

Distributed Systems - 24_09

recupera primi 5 minuti di lezione, causa ritardi treni

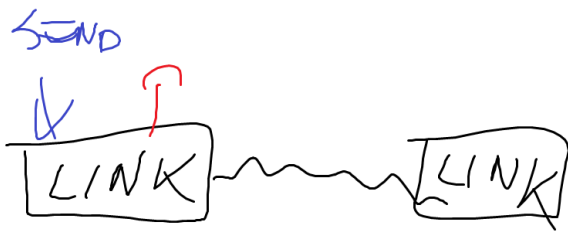
A **cable** is something that loses message with a certain probability Pr . The link may be a cable. In Internet, there is a failure probability.

A channel might duplicate a message a finite number of times, and it does not create messages from thin air.

The description we have just given is an abstraction of a real communication link (an informal description).

Let's take the *Link* object. It takes two kind of events:

- $\langle \text{Send} \mid q, m \rangle$, send a message m to process q
- $\langle \text{Deliver } p, m \rangle$, delivers message m from process p



The probability that the link loses a message pr is known to be $pr < 1$.

Let's define a property:

1. (Fair-loss): if a correct process p sends *infinitely* often m to a process q , then q delivers m and *infinite* number of time.

We use the concept of infinite because we want to get rid of the concept of probability. Probability is difficult to handle with proof.

Let's say for absurd that we want to use it. We have a channel that loses messages with a probability $pr = \frac{1}{2}$. If i send the message 10 time, the probability that the other process receives the message is $1 - \frac{1}{2^{10}}$. We can't write the property taking probability into consideration because we should rewrite the property for each value of probability pr .

By using the concept of infinite I'm creating a property for whatever channel with a probability $pr < 1$. *If I try hard enough, one message will go to the other side.*

Other properties:

- 2) (Finite duplication) If a correct process p sends m a finite number of time to q ,

then q cannot deliver m an infinite number of times.

3) (No creation) If some certain process q delivers a message m with sender p , then m was sent by p to q . *The channel cannot invent something, if q as a link delivers a certain message received by p , then it was specifically sent by p to q .*

This kind of Link is called Fair-Loss link. It has as events and properties the one we've described until now. One link connect you with only one link and one process.

There is a characterization of classes, properties can belong in:

- Safety class: a safety property is a property that if violated at a time t , it can never be satisfied after t . Formally, if a safety property is violated in execution E , there is a prefix E' of E such that any extension of E' also violates the property. *An example of safety property in real life is "i should not die", the moment it happens the property is violated and will be violated forever.*

The Property (3) is a safety property.

Once a process q delivers a message m that was not sent by anyone FL3 is violated forever.

- Liveness class: is a property that cannot be violated in finite execution. Formally, given any finite execution E , there is an extension of E that satisfies it. *informally, a liveness property specify that something good will happen.*

The first one is a liveness property, there is infinite in the property. Imagine that process p send m to a destination, and the destination has not received anything at time t , there is always hope that the destination will receive message in the future at time $t + \Delta$.

Also the second one is a liveness property. In order to not be a liveness property, you should have a finite number of events where the sender sends a message a finite number of times, and the other receives the message an infinite number of times, but it is not possible, because the execution is finite.

- It cannot be violated in finite execution.
 - $E = (\text{del}(p,q,m), \text{del}(p,q,m), \text{del}(p,q,m), \dots, \text{del}(p,q,m))$
 - $E' = (\text{del}(p,q,m), \text{del}(p,q,m), \text{del}(p,q,m), \dots, \text{del}(p,q,m), e_1, e_2, e_3, \dots)$
 - Such that no e_j is $\text{del}(p,q,m)$.

How to violate a program with this property? We create a while true loop.

To violate the first property, in the other hand, is by creating an algorithm that discards received messages.

One property can only belong in one of the classes (mutually exclusive).

Let's now take a look at a variation of a property.

- (At-most7 duplication) If a correct process p sends m a finite number of times to q , then q cannot deliver m more than 7 times.

This property is a safety, we can show an execution that violates this thing in a finite number of times. If I take a liveness property and put a bound, it becomes a safety one.

When we see that in the property there is the concept of infinite, most of the times is a liveness property.

Let's now do a quick exercise on three properties.

- Mutual exclusion if a process p is granted a resource r at time t , then no other process q is granted r at t . This one is safety.

- No-deadlock: if r is not already granted, eventually someone get a grant on resource r .
- No-Starvation: if a process p request a grant on resource r , it will eventually get it.

The second one is a liveness property, we can show an execution where the resource is free for one year, but then someone claims it.

The third one is also liveness. It could be violated with an algorithm.

An example of badly written properties:

- If process p sends a message m to q , then q will eventually deliver it and this deliver is unique.
 - It is mixing two aspects:
 - A liveness: q will eventually deliver it
 - A safety: the deliver is unique

This property could be written, but it should be divided in two properties because it mixes the classes.

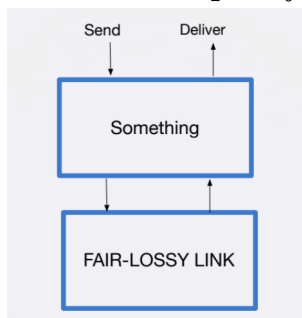
It's not good to mix properties, because when we do proofs we have to use different styles to prove if something is safety or liveness.

- Safety or liveness?
 - Fairness of an execution.
 - (At most- f failures) An execution E can contain at most f failures.
 - (Eventual recovery) If a process p experience a crash(p), then it eventually experiences also a recovery(p) (this event starts again process p).
 - (Time-bounded recovery) If a process p experiences a recovery(p), then this recovery happens at most k events after its failure.
- Write three new safety properties and three new liveness properties.
- Does Time-bounded recovery implies Eventual recovery?
- How would you improve the writing of this property:
 - If a process p outputs a set of integer O_p , then any other process q outputs also outputs a set O_q . For any two processes z and t we have either O_z is a subset of O_t or that O_t is a subset of O_z .

We have seen that async execution must have fairness execution. It's a liveness property.

Algorithms: communication links

We have as a channel the fair-lossy link. It's a box, it interacts with other boxes by means of arrow. Let's say we have a process p that executes with the fair-loss link, and then there is something that does something and interacts with a human. We have to specify this something.



One thing we do not like is the infinite sending of messages. We want to create a channel in which this property is not there. We then want a channel that does not have duplication. We use the fair-lossy to create a link that fixes some of the problems, then another one in top of it that fixes other problems, then we'll have a perfect link.

In the first step we create a link that substitutes the two first properties with only one.

- (Stubborn-delivery) If a correct process p send m to q once, then q delivers m an infinite number of times. [liveness property]

We'll have the stubborn link, *something that does the same thing over and over*.

How do we write the algorithm. We use the reactive computing model, by using pseudo-code. We use events by using handlers, they are functions always acting, and when an event happens and this matches an handler, the handler starts to consume the event.

Handlers are atomic, which means that when a function acts, it lock everything and no other function can act. We can never have two handlers executing in the same process at the same time.

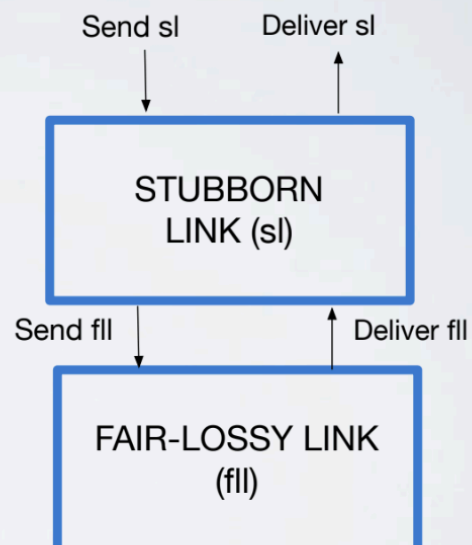
It's possible that an handler does not finish to execute, it happens when the process crashes.

Our stubborn link is given by:

- Events:
 - Requests: $\langle \text{Send} \mid q, m \rangle$ send a message m to process q .
 - Indication: $\langle \text{Deliver}, p, m \rangle$ delivers message m from process p .
- Properties:
 - SL1: (Stubborn-delivery) If a correct process p sends m to q , then q delivers m an infinite number of times.
 - FL3: (No creation) If some process q delivers a message m with sender p , then m was sent by p to q .

- Properties:
 - SL1: (Stubborn-delivery) If a correct process p sends m to q , then q delivers m an infinite number of times.
 - FL3: (No creation) If some process q delivers a message m with sender p , then m was sent by p to q .

```
upon event  $\langle sl, \text{Init} \rangle$  do  
     $sent := \emptyset$ ;  
     $starttimer(\Delta)$ ;  
  
upon event  $\langle \text{Timeout} \rangle$  do  
    forall  $(q, m) \in sent$  do  
        trigger  $\langle fl, \text{Send} \mid q, m \rangle$ ;  
         $starttimer(\Delta)$ ;  
  
upon event  $\langle sl, \text{Send} \mid q, m \rangle$  do  
    trigger  $\langle fl, \text{Send} \mid q, m \rangle$ ;  
     $sent := sent \cup \{(q, m)\}$ ;  
  
upon event  $\langle fl, \text{Deliver} \mid p, m \rangle$  do  
    trigger  $\langle sl, \text{Deliver} \mid p, m \rangle$ ;
```



An *fl* is an instance of a fair-lossy link (*imagine that the link is a class, then the*

fl is an instance of that class).

With upon even we indicate an handler. We say that when the event happens, execute something. With the first we are saying that, when the initialization happens (of the *sl*, instance of the stubborn link), do something. Specifically, we are creating a variable *Sent* that initially is empty $\emptyset$. We start then a timer of Δ , we don't care about the value of Δ .

With the last one we are saying that on event *fl* creates an instance of *Deliver* | *p,m*.

As humans we can create an event, like sending a message *m* to a process *q*. There is a handler that manages this thing. We trigger something, creating a new event pushing on the lower layer, the *fl*. We then take the set *Sent*, and inside I put a set $\{(q,m)\}$. (a reminder that I have to send a message).

In a certain process q, when received a message, it will sent the message to the upper layer once received. The "connection" is always between fair-loss links, and not between stubborn links.

There is still another handler, made for a timeout event. We scan all the messages in the set *Sent*, and we send the messages again, starting another timer again, with the duration of Δ . We need this handler because that this channel is fair-lossy, so we need an infinite loop. *we need to send infinite copy so that the other side receives infinite copies.*

Let's take a look at the first proofs, to show that the algorithm implements properties.

We want to show that it implements FL3 at first. We prove it by contradiction. *we negate the statement we want to prove, supposing it's false, showing that it's not possible to negate it.*

- Suppose process *q* executing our algorithm receives message *m* that was not sent by *p*.
- Fact 1: If *q* delivers a message, then it delivers here:

```
upon event  $\langle fl, Deliver \mid p, m \rangle$  do  
  trigger  $\langle sl, Deliver \mid p, m \rangle$ ;
```

If *q* has a delivery event, than it went trough the fourth trigger, especially the action. If this happened, it means that an handler was triggered by a *Delivery* event in the *fl*. We are assuming that message *m* was invented. It's not possible that *fl* says that *m* was invented, because it would violate FL3. So it's not possible that *m* was invented, and thus FL3 is respected.

- SL1: (Stubborn-delivery) If a correct process p sends m to q , then q delivers m an infinite number of times.
- Proof. By contradiction. Suppose q delivers m a finite number of times.
- Fact 1: p sends m on fll an infinite number of times:

```

upon event { Timeout } do
  forall  $(q, m) \in sent$  do
    trigger {  $fll, Send \mid q, m$  };
    starttimer( $\Delta$ );

upon event {  $sl, Send \mid q, m$  } do
  trigger {  $fll, Send \mid q, m$  };
   $sent := sent \cup \{(q, m)\}$ ;

```

- Fact 2: if q "stubborn delivers" m a finite number of times, then fll delivered m a finite number of times.

```

upon event {  $fll, Deliver \mid p, m$  } do
  trigger {  $sl, Deliver \mid p, m$  };

```

We know that because of the timeout event, the messages will be sent an infinite amount of times.

we follow a reasoning similar to before, by assuming, for example, that the event delivery was only executed 10 times.

Let's see now how to improve stubborn. We want to create something that's easy to use.

- Properties:
 - SL1: (Stubborn-delivery) If a correct process p sends m to q , then q **delivers m an infinite number of times**.
 - FL3: (No creation) If some process q delivers a message m with sender p , then m was sent by p to q .
- In:
 - PL1: (Reliable delivery) If a correct process p sends m to q , then q eventually delivers m .
 - PL2: (No duplication) A message is delivered at most once.
 - FL3: (No creation) If some process q delivers a message m with sender p , then m was sent by p to q .

How to use the stubborn to implement these three properties? We need to create a layer above the stubborn in order to implement these properties. A problem we have to solve is the no duplication. We could see if the received messages are already in memory.

We create the perfect p-2-p algorithm.

```

upon event  $\langle pl, Init \rangle$  do
     $delivered := \emptyset$ ;

```

```

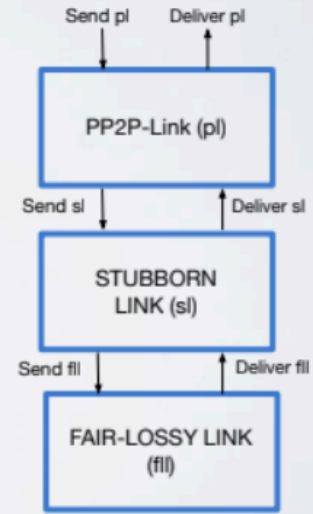
upon event  $\langle pl, Send \mid q, m \rangle$  do
    trigger  $\langle sl, Send \mid q, m \rangle$ ;

```

```

upon event  $\langle sl, Deliver \mid p, m \rangle$  do
    if  $m \notin delivered$  then
         $delivered := delivered \cup \{m\}$ ;
        trigger  $\langle pl, Deliver \mid p, m \rangle$ ;

```



In delivered we save the messages we want to deliver. We check if $m \notin delivered$, if that we save message m in the deliveredset, and then we trigger the deliver event of the pl. We filter infinity with the last handler.